

Human review packet: NoothigattuEtAl2020PairwiseComparisons

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: NoothigattuEtAl2020PairwiseComparisons
Paper id: NoothigattuEtAl2020PairwiseComparisons
Source version: NeurIPS 2020 published conference paper and official supplement
Source record: audit/paper_statement_map.json
Rows: 10
Generated: 2026-09-06
Source URL: [official source](#)

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) AppliedModelingLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-repository-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper NoothigattuEtAl2020PairwiseComparisons --serve
```

Open <http://127.0.0.1:8765/> in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's local reviewer-owned review record.

Contents

- [Governing Lean declarations \(27; supporting context\)](#)
- [Paper-specific semantic prerequisites \(4; not paper claims\)](#)
 - [NoothigattuEtAl2020PairwiseComparisons.definition3_1_pareto_efficiency](#)
 - [NoothigattuEtAl2020PairwiseComparisons.definition4_1_monotonicitySpec](#)
 - [NoothigattuEtAl2020PairwiseComparisons.definition5_1_pairwise_majority_consistencySpec](#)
 - [NoothigattuEtAl2020PairwiseComparisons.definition6_1_separabilitySpec](#)
- [Source claims \(10\)](#)
 - [lemma2_1_mle_existsSpec](#)
 - [lemma2_2_one_neighbor_perfect_fitSpec](#)
 - [lemma2_3_mle_supNorm_boundSpec](#)
 - [lemma2_4_strictConcavity_and_unique_mleSpec](#)
 - [theorem3_2_mle_satisfies_pareto_efficiencySpec](#)
 - [theorem4_2_mle_satisfies_monotonicitySpec](#)
 - [theorem5_3_mle_violates_pairwise_majority_consistencySpec](#)
 - [theorem6_3_mle_violates_separabilitySpec](#)
 - [claimA1_coin_flip_likelihoodSpec](#)
 - [claimC1_strict_crossed_productSpec](#)

Governing Lean declarations

These exact graph-bound declaration bodies are retained by the displayed specifications or reviewed prerequisites. They are supporting context and do not add source-result rows or semantic judgments.

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset

Declaration location: reusable library

Lean declaration body

```
field count:
  {Alternative : Type u_1} →
  AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative → Alternative → Alternative → ℕ

field diagonal_zero:
  ∀ {Alternative : Type u_1} (self : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative)
    (alternative : Alternative), self.count alternative alternative = 0
```

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink

Declaration location: reusable library

Lean declaration body

```
field toFun:
  AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink → ℝ → ℝ

field nonneg:
  ∀ (self : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink) (gap : ℝ),
    0 ≤ self.toFun gap

field le_one:
  ∀ (self : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink) (gap : ℝ),
    self.toFun gap ≤ 1

field monotone:
  ∀ (self : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink), Monotone self.toFun

field complementary:
  ∀ (self : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink) (gap : ℝ),
    self.toFun (-gap) + self.toFun gap = 1

field tendsto_atBot_zero:
  ∀ (self : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink),
    Filter.Tendsto self.toFun Filter.atBot (nhds 0)

field tendsto_atTop_one:
  ∀ (self : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink),
    Filter.Tendsto self.toFun Filter.atTop (nhds 1)
```

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector

Declaration location: reusable library

Lean declaration body

```
type:
  Type u_1 → Type u_1

value:
  fun (Alternative : Type u_1) => Alternative → ℝ
```

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.add

Declaration location: reusable library

Lean declaration body

```
type:
  {Alternative : Type u_1} →
  AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative →
  AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative →
  AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative

value:
  fun {Alternative : Type u_1}
    (firstDataset secondDataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative) =>
  { count := fun (first second : Alternative) => firstDataset.count first second + secondDataset.count first second,
    diagonal_zero :=
      AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.add._proof_1 firstDataset secondDataset }
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.edge

Declaration location: reusable library

Lean declaration body

```
type:
{Alternative : Type u_1} →
AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative → Alternative → Alternative → Prop

value:
fun {Alternative : Type u_1} (dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative)
  (first second : Alternative) =>
  0 < dataset.count first second
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.adjacent

Declaration location: reusable library

Lean declaration body

```
type:
{Alternative : Type u_1} →
AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative → Alternative → Alternative → Prop

value:
fun {Alternative : Type u_1} (dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative)
  (first second : Alternative) =>
  dataset.edge first second v dataset.edge second first
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.edge](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.connectedTo

Declaration location: reusable library

Lean declaration body

```
type:
{Alternative : Type u_1} →
AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative → Alternative → Alternative → Prop

value:
fun {Alternative : Type u_1} (dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative)
  (first second : Alternative) =>
  Relation.ReflTransGen dataset.adjacent first second
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.adjacent](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.hasOnlyNeighbor

Declaration location: reusable library

Lean declaration body

```
type:
{Alternative : Type u_1} →
AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative → Alternative → Alternative → Prop

value:
fun {Alternative : Type u_1} (dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative)
  (first second : Alternative) =>
  first ≠ second ∧
  0 < dataset.count first second + dataset.count second first ∧
  ∀ (other : Alternative),
    other ≠ first → other ≠ second → dataset.count first other = 0 ∧ dataset.count other first = 0
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.hasPairwiseMajorityRanking

Declaration location: reusable library

Lean declaration body

```
type:
{Alternative : Type u_1} →
[inst : Fintype Alternative] →
AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative →
Alternative ≈ Fin (Fintype.card Alternative) → Prop

value:
fun {Alternative : Type u_1} [Fintype Alternative]
  (dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative)
  (ranking : Alternative ≈ Fin (Fintype.card Alternative)) =>
  ∀ (first second : Alternative),
    ranking first < ranking second → dataset.count second first < dataset.count first second
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.hasParetoCountDominance

Declaration location: reusable library

Lean declaration body

```
type:
{Alternative : Type u_1} →
AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative → Alternative → Alternative → Prop

value:
fun {Alternative : Type u_1} (dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative)
  (first second : Alternative) =>
  dataset.count second first < dataset.count first second ∧
  ∀ (other : Alternative),
    other ≠ first →
    other ≠ second →
    dataset.count second other < dataset.count first other ∧
    dataset.count other first < dataset.count other second
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.hasSinglePairCountIncrease

Declaration location: reusable library

Lean declaration body

```
type:
{Alternative : Type u_1} →
AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative →
AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative → Alternative → Alternative → Prop

value:
fun {Alternative : Type u_1}
  (baseline updated : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative)
  (winner loser : Alternative) =>
  baseline.count winner loser < updated.count winner loser ∧
  ∀ (first second : Alternative),
    (first, second) ≠ (winner, loser) → updated.count first second = baseline.count first second
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.isConnected

Declaration location: reusable library

Lean declaration body

```
type:
{Alternative : Type u_1} → AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative → Prop

value:
fun {Alternative : Type u_1} (dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative) =>
  ∀ (first second : Alternative), dataset.connectedTo first second
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.connectedTo](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.isReferenceNormalized

Declaration location: reusable library

Lean declaration body

```
type:
{Alternative : Type u_1} →
  Alternative → AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative → Prop

value:
fun {Alternative : Type u_1} (reference : Alternative)
  (score : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative) =>
  score reference = 0
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.perfectFitDistance

Declaration location: reusable library

Lean declaration body

```
type:
{Alternative : Type u_1} →
  AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative →
  (link : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink) →
  Continuous link.toFun → Alternative → Alternative → ℝ

value:
fun {Alternative : Type u_1} (dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative)
  (link : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink)
  (hcontinuous : Continuous link.toFun) (first second : Alternative) =>
  if hpositive : 0 < dataset.count first second ∧ 0 < dataset.count second first then
    Classical.choose
      (AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.perfectFitDistance._proof_1 dataset link
        hcontinuous first second hpositive)
  else 0
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.maxPerfectFitDistance

Declaration location: reusable library

Lean declaration body

```
type:
{Alternative : Type u_1} →
[Fintype Alternative] →
[DecidableEq Alternative] →
AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative →
(link : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink) →
Continuous link.toFun → Alternative → ℝ

value:
fun {Alternative : Type u_1} [Fintype Alternative] [DecidableEq Alternative]
  (dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative)
  (link : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink)
  (hcontinuous : Continuous link.toFun) (reference : Alternative) =>
let values : Finset ℝ :=
  Finset.image (fun (pair : Alternative × Alternative) => dataset.perfectFitDistance link hcontinuous pair.1 pair.2)
  (Finset.univ xs Finset.univ);
values.max'
(AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.maxPerfectFitDistance._proof_1 dataset link
hcontinuous reference)
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.perfectFitDistance](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.randomUtilityWinProbability

Declaration location: reusable library

Lean declaration body

```
type:
{Alternative : Type u_1} →
(ℝ → ℝ) →
AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative →
Alternative → Alternative → ℝ

value:
fun {Alternative : Type u_1} (link : ℝ → ℝ)
  (score : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative)
  (first second : Alternative) =>
link (score first - score second)
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.pairwiseLogLikelihood

Declaration location: reusable library

Lean declaration body

```
type:
{Alternative : Type u_1} →
[Fintype Alternative] →
[DecidableEq Alternative] →
AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative →
(ℝ → ℝ) → AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative → ℝ

value:
fun {Alternative : Type u_1} [Fintype Alternative] [DecidableEq Alternative]
  (dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative) (link : ℝ → ℝ)
  (score : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative) =>
∑ pair ∈ Finset.univ.offDiag,
  ↑(dataset.count pair.1 pair.2) *
  Real.log
  (AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.randomUtilityWinProbability link score pair.1
pair.2)
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.randomUtilityWinProbability](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.isPairwiseMLE

Declaration location: reusable library

Lean declaration body

```
type:
{Alternative : Type u_1} →
[Fintype Alternative] →
[DecidableEq Alternative] →
AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative →
( $\mathbb{R} \rightarrow \mathbb{R}$ ) →
Alternative → AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative → Prop

value:
fun {Alternative : Type u_1} [Fintype Alternative] [DecidableEq Alternative]
(dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative) (link :  $\mathbb{R} \rightarrow \mathbb{R}$ )
(reference : Alternative)
(score : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative) =>
AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.isReferenceNormalized reference score  $\wedge$ 
IsMaxOn (dataset.pairwiseLogLikelihood link)
{candidate : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative |
AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.isReferenceNormalized reference candidate}
score
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.isReferenceNormalized](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.pairwiseLogLikelihood](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.hasUniquePairwiseMLE

Declaration location: reusable library

Lean declaration body

```
type:
{Alternative : Type u_1} →
[Fintype Alternative] →
[DecidableEq Alternative] →
AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative → ( $\mathbb{R} \rightarrow \mathbb{R}$ ) → Alternative → Prop

value:
fun {Alternative : Type u_1} [Fintype Alternative] [DecidableEq Alternative]
(dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative) (link :  $\mathbb{R} \rightarrow \mathbb{R}$ )
(reference : Alternative) =>
 $\exists$  (score : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative),
dataset.isPairwiseMLE link reference score  $\wedge$ 
 $\forall$  (other : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative),
dataset.isPairwiseMLE link reference other  $\rightarrow$  other = score
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.isPairwiseMLE](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.pairwiseMLEMonotonicity

Declaration location: reusable library

Lean declaration body

```
type:
{Alternative : Type u_1} → [Fintype Alternative] → [DecidableEq Alternative] → ( $\mathbb{R} \rightarrow \mathbb{R}$ ) → Prop

value:
fun {Alternative : Type u_1} [Fintype Alternative] [DecidableEq Alternative] (link :  $\mathbb{R} \rightarrow \mathbb{R}$ ) =>
 $\forall$  (baseline updated : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative)
(reference winner loser : Alternative),
baseline.hasSinglePairCountIncrease updated winner loser  $\rightarrow$ 
baseline.hasUniquePairwiseMLE link reference  $\rightarrow$ 
updated.hasUniquePairwiseMLE link reference  $\rightarrow$ 
 $\forall$ 
(baselineScore updatedScore :
```

```

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative),
baseline.isPairwiseMLE link reference baselineScore →
updated.isPairwiseMLE link reference updatedScore →
  ∀ (other : Alternative),
    baselineScore winner - baselineScore other ≤ updatedScore winner - updatedScore other ∧
    updatedScore loser - updatedScore other ≤ baselineScore loser - baselineScore other

```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.hasSinglePairCountIncrease](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.hasUniquePairwiseMLE](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.isPairwiseMLE](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.pairwiseMLESeparable

Declaration location: reusable library

Lean declaration body

```

type:
  (ℝ → ℝ) → Prop

value:
fun (link : ℝ → ℝ) =>
  ∀ {Alternative : Type u_1} [inst : Fintype Alternative] [inst_1 : DecidableEq Alternative]
    (firstDataset secondDataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative)
    (firstReference : Alternative)
    (firstScore : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative)
    (secondReference : Alternative)
    (secondScore : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative)
    (first second : Alternative),
    firstDataset.isPairwiseMLE link firstReference firstScore →
    secondDataset.isPairwiseMLE link secondReference secondScore →
    firstScore second < firstScore first →
    secondScore second < secondScore first →
    ∀ (pooledReference : Alternative)
      (pooledScore : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative),
      (firstDataset.add secondDataset).isPairwiseMLE link pooledReference pooledScore →
      pooledScore second < pooledScore first

```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.add](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.isPairwiseMLE](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.reaches

Declaration location: reusable library

Lean declaration body

```

type:
  {Alternative : Type u_1} →
    AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative → Alternative → Alternative → Prop

value:
fun {Alternative : Type u_1} (dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative)
  (first second : Alternative) =>
  Relation.ReflTransGen dataset.edge first second

```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.edge](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.everyComponentStronglyConnected

Declaration location: reusable library

Lean declaration body

```
type:
{Alternative : Type u_1} → AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative → Prop

value:
fun {Alternative : Type u_1} (dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative) =>
  ∀ (first second : Alternative), dataset.connectedTo first second → dataset.reaches first second
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.connectedTo](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.reaches](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.scoreRespectsPairwiseMajority Ranking

Declaration location: reusable library

Lean declaration body

```
type:
{Alternative : Type u_1} →
[inst : Fintype Alternative] →
  AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative →
  Alternative ≈ Fin (Fintype.card Alternative) → Prop

value:
fun {Alternative : Type u_1} [Fintype Alternative]
  (score : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative)
  (ranking : Alternative ≈ Fin (Fintype.card Alternative)) =>
  ∀ (first second : Alternative), ranking first < ranking second → score second ≤ score first
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.pairwiseMLEPairwiseMajority Consistent

Declaration location: reusable library

Lean declaration body

```
type:
( $\mathbb{R} \rightarrow \mathbb{R}$ ) → Prop

value:
fun (link :  $\mathbb{R} \rightarrow \mathbb{R}$ ) =>
  ∀ {Alternative : Type u_1} [inst : Fintype Alternative] [inst_1 : DecidableEq Alternative]
    (dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative) (reference : Alternative)
    (score : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative)
    (ranking : Alternative ≈ Fin (Fintype.card Alternative)),
    dataset.isPairwiseMLE link reference score →
    dataset.hasPairwiseMajorityRanking ranking →
    AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.scoreRespectsPairwiseMajorityRanking score
    ranking
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.hasPairwiseMajorityRanking](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.isPairwiseMLE](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.scoreRespectsPairwiseMajorityRanking](#)

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.scoreSupNorm

Declaration location: reusable library

Lean declaration body

```
type:
{Alternative : Type u_1} →
[Fintype Alternative] →
[DecidableEq Alternative] →
Alternative → AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative → ℝ

value:
fun {Alternative : Type u_1} [Fintype Alternative] [DecidableEq Alternative] (reference : Alternative)
  (score : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative) =>
  let values : Finset ℝ := Finset.image (fun (alternative : Alternative) => |score alternative|) Finset.univ;
  values.max' (AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.scoreSupNorm._proof_1 reference score)
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector](#)

AppliedModelingLib.coinFlipLogLikelihood

Declaration location: reusable library

Lean declaration body

```
type:
ℝ → ℝ → ℝ → ℝ

value:
fun (heads tails probability : ℝ) => heads * Real.log probability + tails * Real.log (1 - probability)
```

Paper-specific semantic prerequisites

NoothigattuEtAl2020PairwiseComparisons.definition3_1_pareto_efficiency

Paper-local declaration:

NoothigattuEtAl2020PairwiseComparisons.definition3_1_pareto_efficiency

Source locator: cited publication, lines 256-265

Verbatim paper-source connection

Definition 3.1 (Pareto efficiency). Suppose $a, b \in X$ satisfy $\# \{a \succ b\} > \# \{b \succ a\}$, and are such that for every other alternative $x \in X \setminus \{a, b\}$, we have $\# \{a \succ x\} > \# \{b \succ x\}$ and $\# \{x \succ a\} < \# \{x \succ b\}$.

Then, for every MLE $\hat{\mu}$, Pareto efficiency requires that $\hat{\mu}(a) \geq \hat{\mu}(b)$.

To see that this definition captures the desired behavior in the multi-agent case, note that if $a_i \succ b_i$, then the dataset $\#i$ satisfies the condition of Definition 3.1 with high probability as we P_i grow the number of comparisons in $\#i$, and similarly the condition holds for the pooled dataset $\# = \bigcup_i \#i$. This version of Pareto efficiency is feasible; in fact, it is satisfied by most random utility models.

Lean-expanded paper semantic target

type:

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink → Prop

value:

```
fun (link : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink) =>
  ∀ {Alternative : Type} [inst : Fintype Alternative]
    (dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.Alternative) (reference : Alternative)
    (score : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector.Alternative)
    {first second : Alternative},
  have x : DecidableEq Alternative := Classical.decEq Alternative;
  dataset.isPairwiseMLE link.toFun reference score →
  dataset.hasParetoCountDominance first second → score second ≤ score first
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.hasParetoCountDominance](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.isPairwiseMLE](#)

Recorded source-to-declaration screening Verdict: matches

Reason: At 'cited publication:256-265', Definition 3.1 requires every MLE of the fixed random-utility rule to rank 'a' at least as high as 'b' under the displayed strict direct, outgoing, and incoming count comparisons. The expanded Lean target takes the fixed 'link : CDFLikePairwiseLink' as its property parameter, universally quantifies the finite dataset, normalized MLE score, and alternatives, expands 'hasParetoCountDominance' to exactly those three families of strict count inequalities, and concludes 'score second ≤ score first'. Through 'isPairwiseMLE', 'pairwiseLogLikelihood', and 'randomUtilityWinProbability', the same fixed 'link.toFun' governs the MLE. This matches the source property.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

NoothigattuEtAl2020PairwiseComparisons.definition4_1_monotonicitySpec

Paper-local declaration:

NoothigattuEtAl2020PairwiseComparisons.definition4_1_monotonicitySpec

Source locator: cited publication, lines 280-308

Verbatim paper-source connection

Definition 4.1 (Monotonicity). Suppose that $\#$ and $\#$ are two datasets with unique MLEs $\hat{a}$ and $\hat{b}$. Suppose that $\#\{x y\} = \#\{x y\}$ for all $x, y \in X$ except that $\#\{a b\} > \#\{a b\}$. Then, monotonicity requires that for all $x \in X$, $\hat{a} \sim x \wedge \hat{a} \hat{x}$ and $\hat{b} \sim x \wedge [U+F8FF \text{ private-use glyph}] \hat{b} \hat{x}$.

Equivalently, monotonicity requires that if $\#\{a b\}$ decreases, then a becomes weaker relative to other alternatives, and b becomes stronger. We can interpret monotonicity as guaranteeing a kind of participation incentive: If we ask an agent to compare a to b , the agent is assured that the answer can only influence our inferred utilities in the desired direction.

5

[form-feed in source extraction]

```
[U+0001 control character][U+0002 control character][U+0006 control character]
d [U+0001 control character][U+0002 control character][U+0007 control character] d
↪ [U+0001 control character][U+0002 control character][U+0006 control character]
[U+0001 control character][U+0002 control character][U+0003 control character]
[U+0001 control character][U+0002 control character][U+0001 control character] a
↪ [U+0001 control character][U+0002 control character][U+0001 control character]
↪ a
[U+0001 control character][U+0002 control character][U+0004 control character]
-[U+0001 control character][U+0002 control character][U+0007 control character]
↪ -[U+0001 control character][U+0002 control character][U+0006 control character]
↪ c#
[U+0001 control character][U+0002 control character][U+0005 control character]
-[U+0001 control character][U+0002 control character][U+0006 control character] b"
↪ b"
[U+0001 control character][U+0002 control character][U+0001 control character] a -[U+0001 control
↪ character][U+0002 control character][U+0005 control character]
-[U+0001 control character][U+0002 control character][U+0005 control
↪ character]
d
-[U+0001 control character][U+0002 control character][U+0005 control character]
b" -[U+0001 control character][U+0002 control character][U+0004 control character]
↪ c# -[U+0001 control character][U+0002 control character][U+0004 control character]
-[U+0001 control character][U+0002 control character][U+0004 control character]
-[U+0001 control character][U+0002 control character][U+0003 control character]
↪ -[U+0001 control character][U+0002 control character][U+0003 control character]
-[U+0001 control character][U+0002 control character][U+0003 control character] c#
(17, 86, 38, 56, , 4, 19, 89, 1, 67, 83, 34) (95, 74, 36, 53, , 39, 62, 48, 1, 30, 77, 8) (63, 54, 12, 19, , 54, 57, 80, 49, 7, 50, 20)
```

Lean-expanded paper semantic target

type:

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink → Prop

value:

```
fun (link : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink) =>
  ∀ {Alternative : Type} [inst : Fintype Alternative],
    have x : DecidableEq Alternative := Classical.decEq Alternative;
    AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.pairwiseMLEMonotonicity link.toFun
```

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.pairwiseMLEMonotonicity](#)

Recorded source-to-declaration screening Verdict: matches

Reason: At 'cited publication:280-308', Definition 4.1 fixes one MLE rule, assumes two datasets with unique MLEs that differ only by a strict increase in the directed 'a'-over-'b' count, and requires the winner's relative score to weakly increase and the loser's to weakly decrease against every alternative. The expanded Lean target takes the fixed 'link' as its property parameter and expands 'pairwiseMLEMonotonicity' using 'hasSinglePairCountIncrease', 'hasUniquePairwiseMLE', and 'isPairwiseMLE' with that same 'link.toFun'; its two concluding difference inequalities have exactly the source directions. This matches the source property, including uniqueness and all-other-count equality.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

NoothigattuEtAl2020PairwiseComparisons.definition5_1_pairwise_majority_consistencySpec

Paper-local declaration:

NoothigattuEtAl2020PairwiseComparisons.definition5_1_pairwise_majority_consistencySpec

Source locator: cited publication, lines 380-387

Verbatim paper-source connection

Definition 5.1. Suppose it is possible to label alternatives as $X = \{x_1, \dots, x_m\}$ such that whenever $i < j$, it holds that $\#\{x_i \succ x_j\} > \#\{x_j \succ x_i\}$. Then, pairwise majority consistency (PMC)

requires that for every MLE $\hat{x}$, it holds that $\hat{x}_i \succ \hat{x}_j$ for all $i < j$.

In contrast to our previous properties, PMC is violated by random utility models.

Lean-expanded paper semantic target

type:

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink → Prop

value:

fun (link : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink) =>
AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.pairwiseMLEPairwiseMajorityConsistent link.toFun

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.pairwiseMLEPairwiseMajorityConsistent](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Definition 5.1 requires every MLE to weakly respect any complete labeling whose lower-index alternative has a strict direct-count majority over every higher-index alternative. The expanded target applies pairwiseMLEPairwiseMajorityConsistent to the source-class link. Its displayed prerequisites universally quantify finite alternatives, an enumeration by Fin, a dataset, reference-normalized MLE score, and ranking; encode the premise $\text{ranking}(\text{first}) < \text{ranking}(\text{second})$ as $\text{count}(\text{second}, \text{first}) < \text{count}(\text{first}, \text{second})$; and conclude $\text{score}(\text{second}) \leq \text{score}(\text{first})$. Reference normalization selects a representative of the shift-equivalent MLE scores and leaves all required score comparisons unchanged. The displayed Theorem 5.3 use negates this same complete predicate under its stated link hypotheses, so it does not narrow or alter Definition 5.1.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

NoothigattuEtAl2020PairwiseComparisons.definition6_1_separabilitySpec

Paper-local declaration:

NoothigattuEtAl2020PairwiseComparisons.definition6_1_separabilitySpec

Source locator: cited publication, lines 459-467

Verbatim paper-source connection

Definition 6.1. Consider two datasets #1 and #2, and let $\hat{a}_1$ and $\hat{a}_2$ be MLEs. Suppose there exist two alternatives $a, b \in X$ such that $\hat{a}_1 > \hat{b}_1$ and $\hat{a}_2 > \hat{b}_2$. Separability requires that for every MLE $\hat{a}$ for the pooled dataset $\# = \#1 + \#2$, it also holds that $\hat{a} > \hat{b}$.

Separability is also called consistency, and seems particularly desirable in cases where we combine pairwise comparisons from different sources. While perhaps on first glance innocuous, separability is an extremely strong requirement, and few rules satisfy it; one can prove in general that separability constrains rules to be linear [Myerson, 1995]. Since likelihood maximization is not linear, it is no surprise that MLEs for random utility models fail separability.

Lean-expanded paper semantic target

type:

AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink → Prop

value:

fun (link : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink) =>
AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.pairwiseMLESeparable link.toFun

Direct retained dependencies

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.pairwiseMLESeparable](#)

Recorded source-to-declaration screening Verdict: matches

Reason: At 'cited publication:459-467', Definition 6.1 fixes one MLE rule, selects MLEs for two datasets that both rank 'a' strictly above 'b', and requires every MLE of their pooled dataset to preserve that strict ranking. The expanded Lean target takes the fixed 'link' as its property parameter and expands 'pairwiseMLESeparable' so the component and pooled 'isPairwiseMLE' predicates all use that same 'link.toFun'; 'add' is pointwise count addition, the component inequalities are strict, and the pooled score is universally quantified with the same strict conclusion. This matches the source separability property.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Source claims

Review row 1

Source locator: cited publication, lines 207-216

Verbatim source input

Lemma 2.1 (MLE exists). Suppose F is strictly monotonic and continuous. Then the MLE exists if and only if every connected component of the comparison graph $G_\#$ is strongly connected. For alternative $x, y \in X$, we define the perfect-fit distance between x and y as

$$d(x, y) := F \frac{\# \{x \succ y\} - \# \{y \succ x\}}{\# \{x \succ y\} + \# \{y \succ x\}}.$$

This is the difference in utilities of x and y required for the model to exactly match the observed frequencies of $\# \{x \succ y\}$ and $\# \{y \succ x\}$ in the data. We can check that the MLE will respect this perfect-fit distance when an alternative has only a single neighbor in the comparison graph.

[Next verbatim source excerpt]

2 Model

Let X be a finite set of alternatives. For notational simplicity, we let $X^2 = \{(x, y) : x, y \in X, x \neq y\}$ denote the set of distinct pairs of alternatives. Let $\# : X^2 \rightarrow \mathbb{N}$ be a dataset of pairwise comparisons between alternatives: For $x, y \in X$, $\# \{x \succ y\}$ is the number of times x beat y in the dataset. The (pairwise) comparison graph $G_\# = (X, E)$ with respect to dataset $\#$ is the directed graph with the alternatives X as the vertices, and edges E such that there exists a directed edge $(u, v) \in E$ iff $\# \{u \succ v\} > 0$. We say that $G_\#$ is connected if its undirected form is connected, and we call it strongly connected if for all $(x, y) \in X^2$, there is a directed path from x to y in $G_\#$. Given a dataset, our goal is to learn a random utility model (RUM). A random utility model specifies, for any two distinct alternatives $x, y \in X$, the probability that when asking the decision maker to compare x and y , the answer will be $x > y$. (Due to noise, when repeatedly querying the same pair, we may see different answers.) For us, a random utility model is parameterized by a vector $\theta \in \mathbb{R}^X$, where θ_x is an unknown utility value for $x \in X$. When we ask for a comparison between two alternatives $x, y \in X$, we model the decision maker as sampling noisy utilities $u(x)$ and $u(y)$ from distributions parameterized by (and typically centered at) x and y . Then, the decision maker reports the comparison $x > y$ iff $u(x) > u(y)$. In this paper, we focus on random utility models with i.i.d. noise, so that $u(x) = \theta_x + \epsilon(x)$, where $\epsilon(x) \leftarrow P$ is i.i.d. across all alternatives. Let F be the CDF of a random variable which is the difference between two independent random variables with distribution P . Then the probability that alternative x beats y when they are compared is

$$\Pr(x \succ y) = \Pr(u(x) > u(y)) = \Pr(\epsilon(y) - \epsilon(x) < \theta_x - \theta_y) = F(\theta_x - \theta_y). \quad (1)$$

We derived Equation (1) from a specific noise model, but it makes sense for any function $F : \mathbb{R} \rightarrow [0, 1]$ with CDF-like properties, even if it does not correspond to a noise distribution P . Indeed, we can take any F which is non-decreasing, satisfies $F(u) + F(-u) = 1$ for all $u \in \mathbb{R}$, and is such that $\lim_{u \rightarrow -\infty} F(u) = 0$ and $\lim_{u \rightarrow \infty} F(u) = 1$. We adopt Equation (1) as the general definition of a random utility model for our technical results. Two of the most common random utility models are

- the Thurstone-Mosteller (TM) model: We sample utility as $u(x) = \theta_x + \epsilon(x)$, with i.i.d. noise $\epsilon(x) \leftarrow N(0, 1/2)$. This is equivalent to Equation (1) with F as the Gaussian CDF.
- the Bradley-Terry model (equivalent to the Plackett-Luce model restricted to pairwise comparisons), where $\Pr(x \succ y) = \frac{e^{\theta_x}}{e^{\theta_x} + e^{\theta_y}}$. This is Equation (1) with F as the logistic function.

We usually assume that F is strictly log-concave, and that it is strictly monotonic and continuous, so that F has an inverse on $(0, 1)$. These conditions hold for Thurstone-Mosteller and Bradley-Terry.

- 2 We assume P to be a continuous distribution, and so we do not have to worry about ties.
- 3 Continuity of F is guaranteed when the corresponding noise distribution P is continuous.

3

[form-feed in source extraction]

For a random utility model, given a dataset $\#$, our goal is to find parameters $(\theta_x)_{x \in X}$ that best fit $\#$. We find these parameters by maximum likelihood estimation. The log-likelihood is given by

$$L(\theta) = \sum_{(x,y) \in X^2} \# \{x \succ y\} \log F(\theta_x - \theta_y).$$

When the dataset $\#$ is clear from the context, we write $\hat{\theta} \in \mathbb{R}^X$ for a parameter vector that maximizes log-likelihood, and say that $\hat{\theta}$ is the MLE. Note that if $c \in \mathbb{R}$ is a scalar, then $L(\theta + c) = L(\theta) + c \sum_{x \in X} \# \{x \succ x\}$ for all $\theta \in \mathbb{R}^X$ (since $\Pr(x \succ y)$ depends only on the difference $\theta_x - \theta_y$), so the MLE is only defined up to an additive shift. For concreteness, we pick some $r \in X$, call it the reference alternative, and fix $\theta_r = 0$; then, we maximize L over $D = \{\theta \in \mathbb{R}^X : \theta_r = 0\}$. A random utility model is particularly appropriate when the dataset $\#$ consists of pairwise comparisons which are all reported by a single decision maker. However, in many cases the dataset is obtained by pooling reports from many agents, for instance to minimize the labeling effort of each individual agent, or if we have the explicit aim to aggregate preferences from different agents. Some of the axioms we study are explicitly motivated by cases where $\# = \{ \#^i \}_{i \in I}$, i.e., the dataset is obtained by pooling individual datasets, where I is the set of agents. It then seems natural to assume that each agent behaves in accordance with some random utility model with unknown parameters θ^i and unknown CDF-like function F^i . Then the dataset $\#^i$ is generated by repeatedly querying the agent's random utility model for a comparison.

Expanded PaperInterface specification

```
∀ {Alternative : Type u_1} [inst : Fintype Alternative]
(dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative)
(link : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink) (reference : Alternative),
have x : DecidableEq Alternative := Classical.decEq Alternative;
Continuous link.toFun →
  StrictMono link.toFun →
  ((∃ (score : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative),
    dataset.isPairwiseMLE link.toFun reference score) ↔
    dataset.everyComponentStronglyConnected)
```

Retained governing declarations

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.everyComponentStronglyConnected](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.isPairwiseMLE](#)

Lean-elaborated claim roles

```
1. parameter: Type u_1
2. parameter: Fintype Alternative
3. parameter: AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative
4. parameter: AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink
5. parameter: Alternative
6. conclusion: Continuous link.toFun →
  StrictMono link.toFun →
  ((∃ (score : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative),
    dataset.isPairwiseMLE link.toFun reference score) ↔
    dataset.everyComponentStronglyConnected)
```

Lean proof endpoint (built separately)

NoothigattuEtAl2020PairwiseComparisons.lemma2_1_mle_exists_proof

Recorded source-to-Spec screening Verdict: matches

Reason: Source item lemma2_1 at cited publication:207-216 states, in the included finite-alternative, CDF-like-link, fixed-reference MLE context, that for continuous strictly monotone F an MLE exists iff every undirected comparison component is directed strongly connected. The expanded Lean target quantifies a finite Alternative type, dataset, CDF-like link, and reference, assumes Continuous and StrictMono, and asserts exactly the equivalence between existence of a reference-normalized pairwise MLE score and everyComponentStronglyConnected.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 2

Source locator: cited publication, lines 217-223

Verbatim source input

Lemma 2.2. Let F be strictly monotonic and continuous. Suppose that for alternative a there is exactly one alternative b for which $\# \{a \preceq b\} + \# \{b \preceq a\} > 0$. If both $\# \{a \preceq b\} > 0$ and

$\# \{b \preceq a\} > 0$, then any MLE satisfies $a^* \wedge b^* = (a, b)$.

We can use this result to provide a stronger bound on the MLE than the one from Lemma 2.1, which holds under slightly stronger conditions.

Expanded PaperInterface specification

```
∀ {Alternative : Type u_1} [inst : Fintype Alternative]
  (dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative)
  (link : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink) (reference : Alternative)
  (score : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative)
  {first second : Alternative},
  have x : DecidableEq Alternative := Classical.decEq Alternative;
  dataset.isPairwiseMLE link.toFun reference score →
  dataset.hasOnlyNeighbor first second →
  0 < dataset.count first second →
  0 < dataset.count second first →
  Continuous link.toFun →
  StrictMono link.toFun →
  ∃! delta : ℝ,
  link.toFun delta =
    ↑(dataset.count first second) / (↑(dataset.count first second) + ↑(dataset.count second first)) ∧
  score first - score second = delta
```

Retained governing declarations

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.hasOnlyNeighbor](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.isPairwiseMLE](#)

Lean-elaborated claim roles

```
1. parameter: Type u_1
2. parameter: Fintype Alternative
3. parameter: AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative
4. parameter: AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink
5. parameter: Alternative
6. parameter: AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative
7. parameter: Alternative
8. parameter: Alternative
9. conclusion: dataset.isPairwiseMLE link.toFun reference score →
  dataset.hasOnlyNeighbor first second →
  0 < dataset.count first second →
  0 < dataset.count second first →
  Continuous link.toFun →
  StrictMono link.toFun →
  ∃! delta : ℝ,
  link.toFun delta =
    ↑(dataset.count first second) / (↑(dataset.count first second) + ↑(dataset.count second first)) ∧
  score first - score second = delta
```

Lean proof endpoint (built separately)

NoothigattuEtAl2020PairwiseComparisons.lemma2_2_one_neighbor_perfect_fit_proof

Recorded source-to-Spec screening Verdict: matches

Reason: Source item lemma2_2 at cited publication:217-223 assumes continuous strictly monotone F , that a has exactly one comparison neighbor b , and positive counts in both directions, and says every MLE has score difference equal to the perfect-fit inverse value. The expanded Lean target has the same MLE, one-neighbor, two-positive-count, continuity, and strict-monotonicity premises and asserts a unique δ with $F(\delta)$ equal to the observed win fraction and $\text{score}(\text{first}) - \text{score}(\text{second}) = \delta$; under the stated strict monotonicity this is precisely the source's F -inverse formulation.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 3

Source locator: cited publication, lines 224-227

Verbatim source input

Lemma 2.3. Suppose that $\#\{x \mid y\} > 0$ and $\#\{y \mid x\} > 0$ for all x and y , and that F is continuous and strictly monotonic. Then for every MLE $\hat{\mu}$ we have the bound

$$\frac{1}{k} \log \left[\frac{1}{\sum_{(x,y)} F(x,y)} \right] \leq \max(x, y).$$

Expanded PaperInterface specification

```
∀ {Alternative : Type u_1} [inst : Fintype Alternative],
have x : DecidableEq Alternative := Classical.decEq Alternative;
∀ (dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative)
(link : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink)
(reference : Alternative)
(score : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative),
dataset.isPairwiseMLE link.toFun reference score →
(∀ (first second : Alternative), first ≠ second → 0 < dataset.count first second) →
  ∀ (hcontinuous : Continuous link.toFun),
    StrictMono link.toFun →
      AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.scoreSupNorm reference score ≤
        ↑(Fintype.card Alternative) * dataset.maxPerfectFitDistance link hcontinuous reference
```

Retained governing declarations

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.isPairwiseMLE](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.maxPerfectFitDistance](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.scoreSupNorm](#)

Lean-elaborated claim roles

```
1. parameter: Type u_1
2. parameter: Fintype Alternative
3. conclusion: ∀ (dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative)
(link : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink) (reference : Alternative)
(score : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative),
dataset.isPairwiseMLE link.toFun reference score →
(∀ (first second : Alternative), first ≠ second → 0 < dataset.count first second) →
  ∀ (hcontinuous : Continuous link.toFun),
    StrictMono link.toFun →
      AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.scoreSupNorm reference score ≤
        ↑(Fintype.card Alternative) * dataset.maxPerfectFitDistance link hcontinuous reference
```

Lean proof endpoint (built separately)

NoothigattuEtAl2020PairwiseComparisons.lemma2_3_mle_supNorm_bound_proof

Recorded source-to-Spec screening Verdict: matches

Reason: Source item lemma2_3 at cited publication:224-227 gives, for positive counts in both directions on every pair in X^2 and continuous strictly monotone F , the bound $\frac{1}{|X|} \log \left[\frac{1}{\sum_{(x,y)} F(x,y)} \right] \leq \max(x, y)$ times the maximum perfect-fit distance. The queue's model context defines X^2 as distinct ordered pairs, and the expanded Lean target accordingly assumes positive count for every distinct ordered pair, quantifies every MLE, uses scoreSupNorm, and concludes the same $\text{card}(\text{Alternative})$ times maxPerfectFitDistance bound with the same continuity and strict-monotonicity hypotheses.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 4

Source locator: cited publication, lines 239-240

Verbatim source input

Lemma 2.4. Suppose that F is strictly log-concave. Then $L(\cdot)$ is strictly concave and the MLE is unique (assuming it exists), if and only if the comparison graph $G\#$ is connected.

Expanded PaperInterface specification

```
∀ {Alternative : Type u_1} [inst : Fintype Alternative]
(dataset : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative)
(link : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink) (reference : Alternative),
(StrictConcaveOn ℝ Set.univ fun (gap : ℝ) => Real.log (link.toFun gap)) →
have x : DecidableEq Alternative := Classical.decEq Alternative;
(StrictConcaveOn ℝ
  {score : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative |
    AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.isReferenceNormalized reference score}
  (dataset.pairwiseLogLikelihood link.toFun) ∧
  ∀ (score : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative),
    dataset.isPairwiseMLE link.toFun reference score →
    ∀ (other : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative),
      dataset.isPairwiseMLE link.toFun reference other → other = score) ↔
dataset.isConnected
```

Retained governing declarations

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.isConnected](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.isPairwiseMLE](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.isReferenceNormalized](#)
- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.pairwiseLogLikelihood](#)

Lean-elaborated claim roles

```
1. parameter: Type u_1
2. parameter: Fintype Alternative
3. parameter: AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset Alternative
4. parameter: AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink
5. parameter: Alternative
6. assumption: StrictConcaveOn ℝ Set.univ fun (gap : ℝ) => Real.log (link.toFun gap)
7. conclusion: (StrictConcaveOn ℝ
  {score : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative |
    AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.isReferenceNormalized reference score}
  (dataset.pairwiseLogLikelihood link.toFun) ∧
  ∀ (score : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative),
    dataset.isPairwiseMLE link.toFun reference score →
    ∀ (other : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.ScoreVector Alternative),
      dataset.isPairwiseMLE link.toFun reference other → other = score) ↔
dataset.isConnected
```

Lean proof endpoint (built separately)

NoothigattuEtAl2020PairwiseComparisons.lemma2_4_strictConcavity_and_unique_mle_proof

Recorded source-to-Spec screening Verdict: matches

Reason: Source item lemma2_4 at cited publication:239-240 says that under strict log-concavity, likelihood strict concavity together with uniqueness of any existing MLE holds iff the comparison graph is connected. The expanded Lean target assumes StrictConcaveOn of log F, places likelihood strict concavity on the fixed-reference score domain from the source model, expresses 'unique assuming it exists' as equality of any two MLEs without adding existence, and makes exactly that conjunction equivalent to dataset.isConnected.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 5

Source locator: cited publication, lines 266-271

Verbatim source input

Theorem 3.2. Maximum likelihood estimation satisfies Pareto efficiency if F is strictly monotonic.

The key idea behind the proof (given in Appendix C) is that if $a, b \in \mathcal{X}$ satisfy the condition of Definition 3.1 but some MLE $\hat{\theta}$ puts $\hat{\theta}(a) < \hat{\theta}(b)$, then the parameter vector equal to $\hat{\theta}$ except that

$\hat{\theta}(a) = \hat{\theta}(b)$ and $\hat{\theta}(b) = a$ has strictly higher log-likelihood.

Expanded PaperInterface specification

$\forall$ (link : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink),
StrictMono link.toFun $\rightarrow$ NoothigattuEtAl2020PairwiseComparisons.definition3_1_pareto_efficiency link

Retained governing declarations

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink](#)
- [NoothigattuEtAl2020PairwiseComparisons.definition3_1_pareto_efficiency](#)

Lean-elaborated claim roles

1. parameter: AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink
2. assumption: StrictMono link.toFun
3. conclusion: NoothigattuEtAl2020PairwiseComparisons.definition3_1_pareto_efficiency link

Lean proof endpoint (built separately)

NoothigattuEtAl2020PairwiseComparisons.theorem3_2_mle_satisfies_pareto_efficiency_proof

Recorded source-to-Spec screening Verdict: matches

Reason: Source item theorem3_2 at cited publication:266-271 says that for a strictly monotone CDF-like F , maximum likelihood estimation satisfies Definition 3.1. The expanded theorem target universally fixes the CDF-like link, assumes exactly StrictMono link.toFun, and concludes the graph-bound definition3_1_pareto_efficiency for that same link. Its supporting declaration quantifies every finite dataset, fixed-reference MLE score, and alternative pair and, under the exact source count conditions (strict direct majority plus strictly better outgoing and incoming counts against every other alternative), concludes $\text{score}(\text{second}) \leq \text{score}(\text{first})$, matching Definition 3.1 and the source proof synopsis that rules out $\text{score}(\text{first}) < \text{score}(\text{second})$.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 6

Source locator: cited publication, lines 324-371

Verbatim source input

Theorem 4.2. Maximum likelihood estimation satisfies monotonicity if F is strictly monotonic, log-concave, and differentiable.

The proof, given in Appendix D, is relatively unwieldy. For intuition, let us provide an outline of a proof for the special case of three alternatives a, b, c . Let $\#$ and $\#$ be datasets that are identical except that $\#\{a \mid b\} < \#\{a \mid b\}$, and let $\hat{a}$ and $\hat{b}$ be the respective MLEs, which are unique by Lemma 2.4. We take a as reference, so $\hat{a} = \hat{a} = 0$. It is easy to see that $\hat{b} > \hat{b}$, since otherwise $\hat{a}$ would have greater log-likelihood than $\hat{b}$ for the dataset $\#$, as $\hat{a}$ performs better on the a vs b comparisons, and performs no worse on other comparisons by optimality for $\#$. To see that also $\hat{c} > \hat{c}$, consider first the dataset $\#$ and associated log-likelihood $L(b, c)$ (with a fixed to 0). Now, for $x \in \mathbb{R}$, let (x) denote the value of c that maximizes $L(x, c)$, i.e., maximizes likelihood among parameters with $a = 0$ and $b = x$. One can show that, since F is strictly log-concave, (x) is increasing in x . Notice that the number of comparisons between a and b in a dataset does not influence the optimum position of c , once a and b are fixed. Hence, the function is the same whether defined for $\#$ or for $\#$, since they only differ in a vs b comparisons. We have already seen that $\hat{b} > \hat{b}$ [U+F8FF private-use glyph] $\hat{b}$. Since (x) is increasing, we have $\hat{c} = (\hat{b})$ [U+F8FF private-use glyph] $(\hat{b}) = \hat{c}$, proving monotonicity. To visualize monotonicity, consider the three examples in Figure 1. For four alternatives, we generated random datasets by choosing $\#\{x \mid y\}$ uniformly at random between 1 and 100, and picked three examples. In each case, we let $\#\{b \mid c\}$ vary from 0 to 100 (going horizontally from left to right), and show how the MLE of the Thurstone-Mosteller model changes as the number of b vs c comparisons grows; we fix $\hat{a} = 0$ as reference. As predicted by Theorem 4.2, the orange line of $\hat{b}$ is increasing, while the green line of $\hat{c}$ is decreasing. Note that the change in $\#\{b \mid c\}$ can affect other alternatives; in the middle figure, the relative positions of a and d swap.

In the pooled setting $\# = \mathbb{I} \times \mathbb{I}$ of Section 2, where each agent $i \in \mathbb{I}$ is described by a random utility model with parameters θ_i that generates θ_i , a natural notion of monotonicity is this: Suppose we calculate the MLE $\hat{\theta}$ for $\#$ and suppose we increase the utility θ_i for some agent i and some alternative a while keeping all other parameters fixed. Then the updated MLE $\hat{\theta}$ should satisfy $\hat{\theta}_a \geq \hat{\theta}_x$ for all $x \in \mathbb{X}$: the learned utility of a increases relative to other alternatives.

For the Bradley-Terry model, monotonicity is easier to check, since the first-order conditions of likelihood maximization in that model are well-behaved [González-Díaz et al., 2014, Prop. 6.3]; that proof does not generalize to other models.

Assume that F is twice differentiable. Since $\log F$ is strictly concave, its second derivative is strictly negative. A straightforward calculation shows that then $\partial^2 L / \partial c \partial c < 0$ and that $\partial^2 L / \partial c \partial b > 0$. By definition of (x) , $(\partial L / \partial c)(x, (x)) = 0$. Since $\partial^2 L / \partial c \partial b > 0$, the function $\partial L / \partial c$ is increasing in its first argument, and so $(\partial L / \partial c)(x + \epsilon, (x)) > 0$ for all $\epsilon > 0$. Since $\partial^2 L / \partial c \partial c < 0$, the function $\partial L / \partial c$ is decreasing in its second argument, and hence $(x + \epsilon) > (x)$, as desired.

6

[form-feed in source extraction]

Theorem 4.2 implies that random utility models (subject to the theorem's conditions) satisfy this pooled monotonicity notion with high probability, when $\#$ consists of many samples and when the number of comparisons is uniform across pairs. The reason is this: with high probability, the increase of θ_i increases the number of a vs x comparisons in $\#$ for all x . Assuming for now that no other dataset $\#$ and no other pairs in $\#$ are affected, then successively invoking Theorem 4.2 on a x pairs yields the result. Now, with some probability, other parts will be affected, but not too much. Since the MLE is continuous in $\#$ (see Appendix E), this noise will not invalidate monotonicity.

[Next verbatim source excerpt]

Definition 4.1 (Monotonicity). Suppose that $\#$ and $\#$ are two datasets with unique MLEs $\hat{a}$ and $\hat{b}$. Suppose that $\#\{x \mid y\} = \#\{x \mid y\}$ for all $x, y \in \mathbb{X}$ except that $\#\{a \mid b\} > \#\{a \mid b\}$. Then, monotonicity requires that for all $x \in \mathbb{X}$, $\hat{a} \geq \hat{a}_x$ and $\hat{b} \geq \hat{b}_x$ [U+F8FF private-use glyph] $\hat{b} \geq \hat{a}_x$.

Equivalently, monotonicity requires that if $\#\{a \mid b\}$ decreases, then a becomes weaker relative to other alternatives, and b becomes stronger. We can interpret monotonicity as guaranteeing a kind of participation incentive: If we ask an agent to compare a to b , the agent is assured that the answer can only influence our inferred utilities in the desired direction.

5

[form-feed in source extraction]

```
[U+0001 control character][U+0002 control character][U+0006 control character]
d [U+0001 control character][U+0002 control character][U+0007 control character]
↪ [U+0001 control character][U+0002 control character][U+0006 control character]
[U+0001 control character][U+0002 control character][U+0003 control character]
[U+0001 control character][U+0002 control character][U+0001 control character]
↪ [U+0001 control character][U+0002 control character][U+0001 control character]
↪ a
[U+0001 control character][U+0002 control character][U+0004 control character]
-[U+0001 control character][U+0002 control character][U+0007 control character]
↪ -[U+0001 control character][U+0002 control character][U+0006 control character]
↪ c#
[U+0001 control character][U+0002 control character][U+0005 control character]
-[U+0001 control character][U+0002 control character][U+0006 control character]
↪ b"
[U+0001 control character][U+0002 control character][U+0001 control character]
↪ character][U+0002 control character][U+0005 control character] a -[U+0001 control
-[U+0001 control character][U+0002 control character][U+0005 control character]
↪ character]
d
-[U+0001 control character][U+0002 control character][U+0005 control character]
```

```

      b" -[U+0001 control character][U+0002 control character][U+0004 control character]
      ↪ c# -[U+0001 control character][U+0002 control character][U+0004 control character]
-[U+0001 control character][U+0002 control character][U+0004 control character]
      -[U+0001 control character][U+0002 control character][U+0003 control character]
      ↪ -[U+0001 control character][U+0002 control character][U+0003 control character]
-[U+0001 control character][U+0002 control character][U+0003 control character] c#
      (17, 86, 38, 56, , 4, 19, 89, 1, 67, 83, 34) (95, 74, 36, 53, , 39, 62, 48, 1, 30, 77, 8) (63, 54, 12, 19, , 54, 57, 80, 49, 7, 50, 20)

```

Expanded PaperInterface specification

$\forall$ (link : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink),
 StrictMono link.toFun $\rightarrow$
 (ConcaveOn $\mathbb{R}$ Set.univ fun (gap : $\mathbb{R}$) => Real.log (link.toFun gap)) $\rightarrow$
 Differentiable $\mathbb{R}$ link.toFun $\rightarrow$ NoothigattuEtAl2020PairwiseComparisons.definition4_1_monotonicitySpec link

Retained governing declarations

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink](#)
- [NoothigattuEtAl2020PairwiseComparisons.definition4_1_monotonicitySpec](#)

Lean-elaborated claim roles

1. parameter: AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink
2. assumption: StrictMono link.toFun
3. assumption: ConcaveOn $\mathbb{R}$ Set.univ fun (gap : $\mathbb{R}$) => Real.log (link.toFun gap)
4. assumption: Differentiable $\mathbb{R}$ link.toFun
5. conclusion: NoothigattuEtAl2020PairwiseComparisons.definition4_1_monotonicitySpec link

Lean proof endpoint (built separately)

NoothigattuEtAl2020PairwiseComparisons.theorem4_2_mle_satisfies_monotonicity_proof

Recorded source-to-Spec screening Verdict: matches

Reason: Source item theorem4_2 at cited publication:324-371 says that MLE for a strictly monotone, log-concave, differentiable CDF-like F satisfies Definition 4.1. The expanded theorem target has exactly StrictMono link.toFun, non-strict ConcaveOn of log(link.toFun), and Differentiable link.toFun, then concludes the graph-bound definition4_1_monotonicitySpec for the same fixed link. That supporting property quantifies finite alternatives and expands through pairwiseMLEMonotonicity: the updated dataset changes only the directed winner-loser count and increases it strictly, both datasets have unique fixed-reference MLEs, and their MLE scores satisfy the source's weak winner-up and loser-down difference inequalities relative to every alternative. Thus the dependency and theorem preserve the full source scope without adding uniqueness as an external theorem hypothesis.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 7

Source locator: cited publication, lines 406-416

Verbatim source input

Theorem 5.3. Maximum likelihood estimation violates pairwise majority consistency whenever F is strictly monotonic, strictly log-concave, and differentiable.

How frequent are PMC violations? Write $\frac{\#\{x \succ y\}}{\#\{x \succ y\} + \#\{y \succ x\}}$ for the fraction of x vs y comparisons that x wins. For $X = \{a, b, c\}$, let T be the space of datasets with $0.5 < \frac{\#\{a \succ b\}}{\#\{a \succ b\} + \#\{b \succ a\}}, \frac{\#\{a \succ c\}}{\#\{a \succ c\} + \#\{c \succ a\}}, \frac{\#\{b \succ c\}}{\#\{b \succ c\} + \#\{c \succ b\}} \in [U+F8FF \text{ private-use glyph}] 1$.

For all datasets in T , PMC requires that $\hat{a} > \hat{b} > \hat{c}$. In Figure 2, we draw the cube T and show the regions where the MLE for Thurstone–Mosteller fails PMC. Example 5.2, suitably normalized, falls in the upper orange region. Sampling uniformly over T , we find that Thurstone–Mosteller fails PMC in 17.8% of datasets, while Bradley–Terry fails in 16.6% of datasets.

[Next verbatim source excerpt]

Definition 5.1. Suppose it is possible to label alternatives as $X = \{x_1, \dots, x_m\}$ such that whenever $i < j$, it holds that $\frac{\#\{x_i \succ x_j\}}{\#\{x_i \succ x_j\} + \#\{x_j \succ x_i\}} > \frac{\#\{x_j \succ x_i\}}{\#\{x_j \succ x_i\} + \#\{x_i \succ x_j\}}$. Then, pairwise majority consistency (PMC)

requires that for every MLE, it holds that $x_i \succ x_j$ for all $i < j$.

In contrast to our previous properties, PMC is violated by random utility models.

Expanded PaperInterface specification

```
∀ (link : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink),
  StrictMono link.toFun →
  (StrictConcaveOn ℝ Set.univ fun (gap : ℝ) => Real.log (link.toFun gap)) →
  Differentiable ℝ link.toFun →
  ¬NoothigattuEtAl2020PairwiseComparisons.definition5_1_pairwise_majority_consistencySpec link
```

Retained governing declarations

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink](#)
- [NoothigattuEtAl2020PairwiseComparisons.definition5_1_pairwise_majority_consistencySpec](#)

Lean-elaborated claim roles

1. parameter: AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink
2. assumption: StrictMono link.toFun
3. assumption: StrictConcaveOn ℝ Set.univ fun (gap : ℝ) => Real.log (link.toFun gap)
4. assumption: Differentiable ℝ link.toFun
5. conclusion: ¬NoothigattuEtAl2020PairwiseComparisons.definition5_1_pairwise_majority_consistencySpec link

Lean proof endpoint (built separately)

NoothigattuEtAl2020PairwiseComparisons.theorem5_3_mle_violates_pairwise_majority_consistency_proof

Recorded source-to-Spec screening Verdict: matches

Reason: Theorem 5.3 states that maximum-likelihood estimation violates pairwise majority consistency whenever the source link F is strictly monotonic, strictly log-concave, and differentiable. The expanded Lean target universally quantifies a CDFLikePairwiseLink, assumes StrictMono for F , StrictConcaveOn all real gaps for log composed with F , and differentiability of F , then concludes the negation of definition5_1_pairwise_majority_consistencySpec. The displayed link structure supplies the governing random-utility CDF conditions. The displayed Definition 5.1 closure universally quantifies finite alternatives, datasets, reference-normalized MLE scores, and complete labelings with strict direct-count majorities, and requires the corresponding weak score order. Its negation therefore says exactly that this MLE rule has a counterexample to PMC under the theorem's link conditions. Reference normalization only fixes the additive score representative and does not change majority or score ordering. No source premise, quantifier, or conclusion is added, omitted, or narrowed.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 8

Source locator: cited publication, lines 485-496

Verbatim source input

Theorem 6.3. Maximum likelihood estimation violates separability whenever F is strictly monotonic, strictly log-concave, and differentiable.

Like for PMC, we can again ask on what percentage of (pairs of) datasets the MLE fails separately. Since we sample over pairs, we might guess the answer to be of lower order than in the case of PMC, and this is borne out by the data. For $m = 3$ alternatives, sampling uniformly over the space of datasets for which each pair of distinct alternatives is compared equally often, we find that on about 1.5% of dataset pairs, Thurstone–Mosteller fails separability. This fraction increases as m increases, since there are more pairs of alternatives for which separability can be violated.

8

[Next verbatim source excerpt]

Definition 6.1. Consider two datasets $\#1$ and $\#2$, and let $\hat{a}_1$ and $\hat{a}_2$ be MLEs. Suppose there exist two alternatives $a, b \in X$ such that $\hat{a}_1 > \hat{b}_1$ and $\hat{a}_2 > \hat{b}_2$. Separability requires that for every MLE $\hat{a}$ for the pooled dataset $\# = \#1 + \#2$, it also holds that $\hat{a} > \hat{b}$.

Separability is also called consistency, and seems particularly desirable in cases where we combine pairwise comparisons from different sources. While perhaps on first glance innocuous, separability is an extremely strong requirement, and few rules satisfy it; one can prove in general that separability constrains rules to be linear [Myerson, 1995]. Since likelihood maximization is not linear, it is no surprise that MLEs for random utility models fail separability.

Expanded PaperInterface specification

```
∀ (link : AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink),
  StrictMono link.toFun →
    (StrictConcaveOn ℝ Set.univ fun (gap : ℝ) => Real.log (link.toFun gap)) →
      Differentiable ℝ link.toFun → ¬NoothigattuEtAl2020PairwiseComparisons.definition6_1_separabilitySpec link
```

Retained governing declarations

- [AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink](#)
- [NoothigattuEtAl2020PairwiseComparisons.definition6_1_separabilitySpec](#)

Lean-elaborated claim roles

1. parameter: `AppliedModelingLib.Learning.HumanFeedback.PairwiseCountDataset.CDFLikePairwiseLink`
2. assumption: `StrictMono link.toFun`
3. assumption: `StrictConcaveOn ℝ Set.univ fun (gap : ℝ) => Real.log (link.toFun gap)`
4. assumption: `Differentiable ℝ link.toFun`
5. conclusion: `¬NoothigattuEtAl2020PairwiseComparisons.definition6_1_separabilitySpec link`

Lean proof endpoint (built separately)

`NoothigattuEtAl2020PairwiseComparisons.theorem6_3_mle_violates_separability_proof`

Recorded source-to-Spec screening Verdict: matches

Reason: Source item `theorem6_3` at cited publication:485-496 says that every strictly monotone, strictly log-concave, differentiable CDF-like F yields an MLE-rule violation of Definition 6.1 separability. The expanded theorem target universally fixes that `link`, assumes exactly `StrictMono link.toFun`, `StrictConcaveOn` of `log(link.toFun)`, and `Differentiable link.toFun`, and negates the graph-bound `definition6_1_separabilitySpec` for the same `link`. The supporting property expands through `pairwiseMLESeparable` to two datasets and their MLEs under that `link`, assumes both strictly rank the same pair, pointwise-pools their counts, and requires every pooled-data MLE to preserve the strict ordering. This is precisely the source definition, so its negation matches the stated theorem violation.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 9

Source locator: cited publication, lines 780-784

Verbatim source input

Claim A.1 (Coin flip likelihood). For $h, t > 0$ and $p \in (0, 1)$, the function $f(p) = h \cdot \log(p) + t \cdot \log(1-p)$ is strictly concave with the maximum uniquely attained at

$$\hat{p} = \frac{h}{h+t}$$

Expanded PaperInterface specification

```
∀ {heads tails : ℝ},
  0 < heads →
  0 < tails →
  StrictConcaveOn ℝ (Set.Ioo 0 1) (AppliedModelingLib.coinFlipLogLikelihood heads tails) ∧
  (heads / (heads + tails) ∈ Set.Ioo 0 1) ∧
  IsMaxOn (AppliedModelingLib.coinFlipLogLikelihood heads tails) (Set.Ioo 0 1) (heads / (heads + tails)) ∧
  ∀ maximizer ∈ Set.Ioo 0 1,
  IsMaxOn (AppliedModelingLib.coinFlipLogLikelihood heads tails) (Set.Ioo 0 1) maximizer →
  maximizer = heads / (heads + tails)
```

Retained governing declarations

- [AppliedModelingLib.coinFlipLogLikelihood](#)

Lean-elaborated claim roles

```
1. parameter: ℝ
2. parameter: ℝ
3. assumption: 0 < heads
4. assumption: 0 < tails
5. conclusion: StrictConcaveOn ℝ (Set.Ioo 0 1) (AppliedModelingLib.coinFlipLogLikelihood heads tails) ∧
  (heads / (heads + tails) ∈ Set.Ioo 0 1) ∧
  IsMaxOn (AppliedModelingLib.coinFlipLogLikelihood heads tails) (Set.Ioo 0 1) (heads / (heads + tails)) ∧
  ∀ maximizer ∈ Set.Ioo 0 1,
  IsMaxOn (AppliedModelingLib.coinFlipLogLikelihood heads tails) (Set.Ioo 0 1) maximizer →
  maximizer = heads / (heads + tails)
```

Lean proof endpoint (built separately)

NoothigattuEtAl2020PairwiseComparisons.claimA1_coin_flip_likelihood_proof

Recorded source-to-Spec screening Verdict: matches

Reason: Source item claimA_1 at cited publication:780-784 states that for positive real h and t , the displayed coin-flip log-likelihood is strictly concave on p in $(0,1)$ and has its unique maximum at $h/(h+t)$. The expanded Lean target has exactly the two positivity assumptions, StrictConcaveOn on Set.Ioo 0 1, membership and IsMaxOn at heads/(heads+tails), and equality of every other in-domain maximizer to that point, so its scope and conclusion coincide with the source item.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 10

Source locator: cited publication, lines 1055-1055

Verbatim source input

Claim C.1. Let $c, d, e, f > 0$ such that $c > d$ and $e > f$. Then $ce + df > cf + de$.

Expanded PaperInterface specification

$\forall \{c\ d\ e\ f : \mathbb{R}\}, 0 < c \rightarrow 0 < d \rightarrow 0 < e \rightarrow 0 < f \rightarrow d < c \rightarrow f < e \rightarrow c * e + d * f > c * f + d * e$

Lean-elaborated claim roles

- 1. parameter: $\mathbb{R}$
- 2. parameter: $\mathbb{R}$
- 3. parameter: $\mathbb{R}$
- 4. parameter: $\mathbb{R}$
- 5. assumption: $0 < c$
- 6. assumption: $0 < d$
- 7. assumption: $0 < e$
- 8. assumption: $0 < f$
- 9. assumption: $d < c$
- 10. assumption: $f < e$
- 11. conclusion: $c * e + d * f > c * f + d * e$

Lean proof endpoint (built separately)

NoothigattuEtAl2020PairwiseComparisons.claimC1_strict_crossed_product_proof

Recorded source-to-Spec screening Verdict: matches

Reason: Source item claimC_1 at cited publication:1055-1055 quantifies positive real c,d,e,f , assumes $c>d$ and $e>f$, and concludes $ce+df>cf+de$. The expanded Lean target quantifies the same four reals, lists the same six positivity/order assumptions as $0<c, 0<d, 0<e, 0<f, d<c, f<e$, and has the identical strict crossed-product conclusion.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation